

Vinci Unitree Course — Day 5 Lecture Notes

G1 Architecture, DDS Communication, and Safety Readiness

Duration: 3 hours

Audience: Train-the-trainer cohort preparing to teach Unitree G1 fundamentals

Primary platform: Unitree G1 / G1-EDU humanoid

Course context: Day 5 transitions the course from quadruped inspection workflows to humanoid-system readiness. It deliberately emphasizes **observation, network verification, DDS state streaming, and read-only motion-state inspection** before any locomotion or arm-control commands are attempted.

Vinci AI teaching principle for Day 5: Students must prove that they can observe the G1 safely before they are allowed to command the G1. A robot that can stream state is not automatically safe to move, and a robot that is safe to move must still pass clear network, DDS, mode, and finite-state-machine checks.

1. Day 5 Position in the Course

Day 5 is the bridge between the quadruped inspection autonomy developed earlier in the week and the humanoid command work that follows. Days 2 through 4 focused on inspection evidence, patrol plans, B2 camera workflows, SportModelState logging, supervised motion, and field-run reporting. Day 5 changes the teaching emphasis from **mission execution** to **humanoid readiness discipline**. The G1 has a different morphology, risk profile, network setup, message namespace, and operational posture from Go2 and B2, so students should not assume that previously successful quadruped commands can be transferred directly to the humanoid platform.

The repository Day 5 materials frame the day around **G1 Architecture, DDS & Safety**, with a non-motion-first lab sequence. Lab 0 introduces architecture, environment setup, and safety gates. Lab 1 subscribes to `rt/lowstate` and verifies that high-rate state can be received. Lab 2 performs read-only finite-state-machine and motion-mode checks through SDK clients, preparing students for Day 6 motion labs without prematurely sending locomotion commands. 1

Day 5 theme	What students learn	What students should not do yet
G1 architecture	Distinguish G1 hardware, development computer, control computer, DDS topics, and high-level service clients.	Treat G1 as simply “a bigger Go2” or copy Go2/B2 code without checking IDL types.
Network readiness	Configure the 192.168.123.x subnet, identify the correct network interface, and verify robot reachability.	Pass an IP address where an SDK example expects an interface name.
DDS observation	Subscribe to rt/lowstate , inspect message timing, IMU orientation, motor count, and mode fields.	Publish low-level commands to rt/lowcmd during Day 5.
Read-only motion-state inspection	Query motion-switcher and G1 locomotion FSM readiness without commanding motion.	Use Damp() , Move() , arm commands, or special actions casually.
Safety culture	Apply humanoid-specific spacing, spotter, remote-control, and recovery discipline.	Assume streaming data means the robot is ready to stand, wave, or walk.

The official Unitree G1 documentation describes the robot as a humanoid divided into upper and lower body assemblies, with the G1 listed as 23 degrees of freedom and the G1-EDU as 23–43 degrees of freedom depending on options. ⁵ The product page lists approximately 35 kg weight, 1320 × 450 × 200 mm standing dimensions, depth camera and 3D LiDAR sensing, and secondary development availability on the EDU version. ⁸ These physical facts explain why Day 5 must emphasize distance, posture, and readiness more strongly than the earlier quadruped labs.

2. Learning Outcomes

By the end of the three-hour lecture and lab session, participants should be able to explain the G1 development stack in words, verify the local SDK environment, identify the correct Ethernet interface, subscribe to the G1 low-state topic, and interpret whether the robot is merely reachable, fully streaming, or actually ready for high-level motion. They should also be able to teach the difference between DDS topics and RPC-style SDK clients, and they should be able to explain why Day 5 is intentionally conservative.

Outcome category	Specific competency	Evidence of mastery
Architecture	Explain the difference between the user PC, G1 control computer, G1 development computer, DDS middleware, and high-level service clients.	Student can draw the data path from laptop to G1 and identify where <code>rt/lowstate</code> , <code>rt/lowcmd</code> , and <code>LocoClient</code> belong.
Environment	Verify Python SDK imports, CycloneDDS installation, and session variables such as <code>CYCLONEDDS_HOME</code> .	Student can run the course verification script and explain what it verifies locally versus on the robot.
Networking	Configure and identify a <code>192.168.123.x</code> network adapter and test reachability to the G1.	Student can use <code>ip addr</code> , <code>ifconfig</code> , or equivalent tools to find the correct interface and can ping the robot control IP.
DDS observation	Subscribe to <code>rt/lowstate</code> with the correct G1 IDL type.	Student receives low-state messages and can identify <code>tick</code> , <code>mode_machine</code> , IMU orientation, and motor count.
Read-only readiness	Query motion-switcher mode and finite-state-machine state without commanding motion.	Student can distinguish “DDS healthy” from “motion ready.”
Safety	State the Day 5 no-motion rule and explain recovery and stop expectations.	Student can describe what damped mode means and why it can lead to loss of balance.

3. Three-Hour Teaching Plan

The Day 5 session should be taught as a progressive readiness funnel. The lecture begins with the physical robot and safety context, narrows into network and SDK setup, then proves state streaming, and finally uses read-only motion-state calls to determine whether the robot is ready for later command labs. The instructor should resist the temptation to demonstrate motion early, because the central lesson is that disciplined observation prevents unsafe command behavior.

Time	Segment	Instructor objective	Student activity	Deliverable
------	---------	----------------------	------------------	-------------

0:00–0:15	Opening and safety frame	Establish why humanoid readiness is different from quadruped inspection control.	Identify risk zones, remote-control responsibility, and no-motion boundaries.	Shared safety checklist.
0:15–0:35	G1 hardware and computing architecture	Explain G1 body structure, DOF, sensing, onboard computers, and development network.	Label G1 physical subsystems and computing roles.	Architecture sketch.
0:35–0:55	SDK, CycloneDDS, and communication patterns	Differentiate topic subscription, topic publication, and RPC clients.	Map <code>rt/lowstate</code> , <code>rt/lowcmd</code> , <code>MotionSwitcherClient</code> , and <code>LocoClient</code> .	Communication map.
0:55–1:10	Break and bench readiness	Confirm that robot, remote, spotter, network cable, and laptop are ready.	Students check interface names and physical environment.	Readiness checkpoint.
1:10–1:35	Lab 0 walkthrough	Verify local SDK environment, network interface, ping, and initial robot reachability.	Run setup/verification scripts and record outputs.	Lab 0 evidence.
1:35–2:05	Lab 1 low-state subscription	Teach G1 DDS topic binding and first-message interpretation.	Subscribe to <code>rt/lowstate</code> and inspect selected fields.	Low-state log evidence.
2:05–2:15	Break and synthesis	Compare successful and failed low-state cases.	Diagnose one common failure case.	Troubleshooting notes.
2:15–2:45	Lab 2 read-only FSM inspection	Use motion-switcher and LocoClient GET	Query current mode and FSM state.	FSM readiness report.

		APIs without sending motion.		
2:45–3:00	Debrief, assessment, and Day 6 preview	Connect Day 5 readiness to Day 6 motion and arm control.	Complete knowledge checks and instructor sign-off.	Day 5 pass/follow-up decision.

4. Safety First: Why Day 5 Is a No-Motion-First Day

The G1 is not just another robot in the course sequence; it is a tall humanoid with a higher center of mass, complex whole-body actuation, and a much greater consequence of falling. Unitree’s G1 product page explicitly cautions that humanoid robots have complex structure and powerful actuation and that users should keep sufficient safe distance. ⁸ The G1 quick-start documentation also states that emergency damped mode can cause the robot to lose balance and fall. ⁶ These details must shape classroom procedure.

Day 5’s practical safety rule is simple: **observe before command, verify before motion, and never confuse damped mode with a harmless pause.** In a quadruped, entering damping or stopping motion may often look like a stable low posture. In a humanoid, removing active balance can result in a fall if the robot is standing without support. Therefore, commands that affect actuation state must be treated as operationally significant, even if their names sound benign.

Instructor warning: Do not use `Damp()` as a casual “stop button” during a standing humanoid demonstration. On G1, damping can remove the active control needed to remain upright. The remote-control operator and physical spotter must understand the intended recovery posture before any transition.

Safety item	Day 5 expectation	Why it matters
Clear working area	Keep students outside the fall radius and remove loose obstacles.	A humanoid fall can travel outward and can damage both people and equipment.
Remote-control owner	Assign one trained person to hold the controller and call out state changes.	State transitions must be coordinated, not improvised by multiple students.
Spotter role	Use a spotter only according to the approved procedure; do not grab moving joints.	Human intervention can become dangerous if performed near actuated limbs.

No-motion default	Labs 0–2 do not send walking, waving, arm, or low-level joint commands.	The learning objective is readiness, not performance.
Damped-mode awareness	Treat damping as a potentially falling state.	Damping can remove balance support and should be entered deliberately.
SDK conflict avoidance	Ensure the correct robot mode before SDK use.	Unitree warns that SDK commands can conflict with built-in motion-control behavior if the robot is not in the intended mode. 6
Evidence logging	Record checks before advancing to motion labs.	Readiness should be auditable, not based on memory.

The G1 quick-start page describes debug mode and explains that SDK development should be performed with the robot in the appropriate mode to avoid conflicting instructions from the built-in motion-control program. 6 For teaching, this means the instructor should separate **physical startup**, **debug/development readiness**, **DDS observation**, and **motion readiness** as four different states rather than compressing them into one vague phrase such as “the robot is connected.”

5. G1 Hardware and Development Context

The G1 is a humanoid research and education platform. Unitree describes the G1 as having 23 degrees of freedom in the base configuration and 23–43 degrees of freedom for the G1-EDU depending on options such as additional waist, wrist, and dexterous-hand degrees of freedom. 5 The product page lists the robot as approximately 35 kg with about two hours of battery life and EDU support for secondary development. 8

The important teaching point is that G1’s morphology changes both the **control problem** and the **classroom protocol**. A quadruped distributes support across four legs and can often return to a stable crouched posture. A humanoid depends on bipedal balance, coordinated whole-body control, and careful state transitions. Therefore, students must learn to inspect modes and telemetry before they reason about commands.

Platform aspect	G1 / G1-EDU relevance to Day 5
Degrees of freedom	Higher DOF means more complex state interpretation and greater need for correct IDL bindings.

Bipedal balance	Standing stability depends on active control, so mode changes can have immediate physical consequences.
Sensing	Official materials list depth camera and 3D LiDAR, but Day 5 focuses on DDS state, not perception algorithms. 8
Development computer	Official G1 developer documentation lists a development computing unit and notes that the control computing unit is dedicated to Unitree motion control. 5
Network topology	Students must connect through the correct network segment and pass the SDK a network interface name.
SDK split	G1 examples and message types are not identical to Go2/B2 examples; the IDL namespace matters.

Unitree’s G1 developer documentation identifies the EDU development computing unit as a Jetson Orin NX and lists the development unit IP as 192.168.123.164 , while the quick-development guide instructs users to configure the user computer on the 192.168.123.x subnet and test connectivity to 192.168.123.161 . 5 7 The course field guide follows this practical approach: students should identify the correct Ethernet adapter, verify subnet membership, and test ping before diagnosing SDK-level problems.

6. The Day 5 Communication Model

Day 5 should be taught using a three-lane communication model: **subscribe**, **publish**, and **request/response**. DDS topic subscription is how students observe high-rate robot state. DDS topic publication is how low-level command messages could be sent, but Day 5 deliberately avoids low-level command publication. RPC-style clients such as MotionSwitcherClient and LocoClient are used to query and, in later labs, potentially control high-level robot services.

Communication lane	Day 5 example	Direction	Day 5 use	Risk level
DDS topic subscription	rt/lowstate	Robot to laptop	Required for Lab 1. Students observe state.	Low, because it is read-only.

DDS topic publication	rt/lowcmd	Laptop to robot	Not used for Day 5 teaching.	High, because it can affect joints at low level.
RPC service client	MotionSwitcherClient.CheckMode()	Laptop to robot service	Used read-only in Lab 2.	Low when only checking mode.
RPC service client	G1 LocoClient GET APIs	Laptop to robot service	Used read-only to inspect FSM and readiness.	Low when GET-only; higher when SET/action calls are introduced later.

The upstream `unitree_sdk2_python` project explains that the Python SDK provides robot status acquisition and control through request-response operations or topic subscription/publishing, and that example programs include high-level status/control, low-level status/control, wireless controller status, front camera, obstacle avoidance, and other services. ⁹ Day 5 uses only the safest subset of that pattern: local SDK verification, DDS low-state subscription, and read-only service checks.

A useful instructor analogy is to compare the robot to an aircraft cockpit. Lab 1 teaches students to read the instruments. Lab 2 teaches students to ask the avionics what mode the aircraft is in. Neither lab authorizes takeoff. The skill being built is operational judgment, not command excitement.

7. Environment Setup and Verification

The official Python SDK README lists Python 3.8 or later, CycloneDDS 0.10.2, NumPy, and OpenCV as dependencies for `unitree_sdk2_python`. ⁹ The Day 5 course materials use setup and verification scripts to reduce installation ambiguity. Students should understand what each script proves and what it does not prove.

Check	What it proves	What it does not prove
Python import check	The local Python environment can import the required SDK modules.	The laptop can reach the robot.
CycloneDDS presence	The middleware dependency can be found by the SDK.	The network interface is correct.
Local DDS hello-world	DDS can work locally on the machine.	Robot topics are visible.

Ping to robot IP	Ethernet path to the expected robot address exists.	DDS multicast and SDK topics are working.
<code>rt/lowstate</code> subscription	The robot is streaming state to the laptop.	The robot is safe or ready to move.
<code>CheckMode()</code> and FSM GET	Read-only service calls can inspect high-level motion readiness.	The next command will be safe unless the physical environment and state are also correct.

The upstream SDK README notes that a common installation error is `Could not locate cyclonedds`, and it instructs users to compile CycloneDDS and set `CYCLONEDDS_HOME` when required. ⁹ In class, this should be framed as a **deterministic dependency problem**, not a robot problem. If imports fail before the network is touched, students should fix the local environment first rather than repeatedly rebooting the robot.

A typical Day 5 shell preparation sequence is:

Bash

```
# Example session preparation; adapt paths to the course machine.
cd ~/vinci-unitree
export CYCLONEDDS_HOME="$HOME/cyclonedds/install"
unset CYCLONEDDS_URI
python3 scripts/verify_install.py
```

The command `unset CYCLONEDDS_URI` is important in classroom settings because a stale CycloneDDS XML profile can silently force DDS onto the wrong interface or discovery configuration. The teaching point is not that `CYCLONEDDS_URI` is bad; rather, students should know whether they are using an explicit DDS profile or the course default behavior.

8. Network Readiness: Interface Name, Not Just IP Address

Unitree's G1 quick-development guide recommends connecting the user computer and the G1 switch to the same network, configuring the user computer in the `192.168.123.x` subnet, and testing connectivity to `192.168.123.161`. ⁷ This is consistent with the general Unitree SDK quick-start guidance, which stresses that the SDK examples need the network-card name corresponding to the configured robot subnet. ¹⁰

The common student mistake is to pass `192.168.123.161` to a script that expects something like `enp3s0`, `eth0`, or `enxf8e43b808e06`. The robot IP identifies the remote endpoint for ping,

but many SDK examples initialize DDS using the **local network interface name**. The instructor should make this distinction explicit and test it verbally.

Term	Example	Meaning	How students verify it
Robot control IP	192.168.123.161	Address commonly used to test connectivity to the robot control computer.	ping 192.168.123.161
G1 development computer IP	192.168.123.164	Address listed in official documentation for the G1 development computing unit. 5	Network documentation and lab-specific needs.
User PC interface IP	192.168.123.99 or 192.168.123.222	Address assigned to the laptop Ethernet adapter on the robot subnet. 7	ip addr or ifconfig
User PC interface name	enp2s0 , eth0 , or USB-Ethernet name	The argument many SDK scripts require.	Match the interface that owns the 192.168.123.x address.

A robust network check sequence for students is:

Bash

```
ip addr
ping -c 3 192.168.123.161
python3 course/day-05/lab-00/g1_connection_check.py <network_interface>
```

The instructor should require students to copy the exact interface name into their lab notes. This prevents a later failure pattern in which students say “I used the robot IP” when the SDK initialization actually needed the interface.

9. Lab 0 Lecture Notes: Architecture, Environment, and Readiness Gate

Lab 0 is the safety and infrastructure gate for the entire G1 sequence. It should not be rushed. Its purpose is to show that the student’s laptop, Python environment, middleware

dependency, Ethernet interface, and robot-reachability path are all plausible before the class attempts to read state or inspect motion modes.

The lab can be introduced with the following statement: **Lab 0 does not prove that G1 can move; it proves that the classroom has a controlled basis for observation.** This distinction protects the class from over-interpreting a green check mark.

Lab 0 checkpoint	Instructor explanation	Pass evidence	Failure interpretation
SDK import	Python can locate the Unitree SDK modules.	Import or verification script succeeds.	Fix virtual environment, package path, or installation.
CycloneDDS found	SDK can find DDS middleware.	No <code>cyclonedds</code> location error.	Set <code>CYCLONEDDS_HOME</code> or repair installation.
Interface identified	Student knows the local adapter connected to G1.	Interface owns a <code>192.168.123.x</code> address.	Configure Ethernet or choose correct adapter.
Ping succeeds	Basic IP path exists.	Replies from <code>192.168.123.161</code> .	Check cable, subnet, robot state, adapter, firewall.
Optional multicast/DDS check	Discovery path is plausible.	DDS test or course script reports success.	Inspect DDS profile and network binding.
Initial lowstate readiness	Robot may be publishing state.	Later confirmed in Lab 1.	Do not proceed to motion; diagnose observation first.

During Lab 0, the instructor should repeatedly ask students whether each test is **local**, **network**, **DDS**, or **robot-service** evidence. This vocabulary builds the troubleshooting discipline needed for Day 6.

10. Lab 1 Lecture Notes: Subscribing to `rt/lowstate`

Lab 1 is the first true robot-observation lab. The course script subscribes to the G1 low-state topic using the G1/HG IDL type, typically imported as `unitree_sdk2py.idl.unitree_hg.msg.dds_.LowState_`. The script initializes the communication factory with the selected interface unless an explicit CycloneDDS profile is already in use, creates a subscriber for `rt/lowstate`, and waits for the first message.

The key teaching point is that **topic name and message type must match the robot family**. Go2 and B2 examples commonly use `unitree_go` message definitions, while G1 uses `unitree_hg` in the Day 5 low-state script. A student who copies a quadruped subscriber may bind the wrong IDL type even if the topic string looks familiar.

Low-state field or evidence	What it means pedagogically	What students should record
First message received	DDS discovery and subscription are functioning.	Time to first message and script output.
<code>tick</code>	The robot is publishing a changing state sequence.	Confirm that values change over time.
<code>mode_machine</code>	Machine-state indicator useful for readiness interpretation.	Current value and whether it matches expected state.
<code>mode_pr</code>	Additional mode/status field exposed by low state.	Current value, if printed.
IMU roll-pitch-yaw	Orientation estimate, useful for posture awareness.	Values and whether they appear physically plausible.
Motor count	Confirms the size of the joint-state array visible in the message.	Count and whether it matches expected G1 configuration.
Message rate	Indicates whether data is streaming continuously.	Approximate update rate or interval.

A simplified conceptual version of the Lab 1 subscriber is:

Python

```
from unitree_sdk2py.core.channel import ChannelFactoryInitialize, ChannelSubscriber
from unitree_sdk2py.idl.unitree_hg.msg.dds_ import LowState_

ChannelFactoryInitialize(0, network_interface)
subscriber = ChannelSubscriber("rt/lowstate", LowState_)
subscriber.Init(callback, 10)
```

This code should be explained in layers. `ChannelFactoryInitialize` binds the SDK/DDS layer to the communication context. `ChannelSubscriber` declares the topic and message type. The callback is where state becomes a teaching object rather than a stream of raw bytes. Students should not simply run the script and move on; they should identify which output line proves first-message success and which fields show ongoing publication.

Teaching prompt: If `ping` succeeds but `rt/lowstate` does not arrive, which layer is likely failing: physical Ethernet, IP routing, DDS discovery, topic/message binding, or robot service state? Students should answer with evidence, not guesses.

11. Interpreting Low-State Without Overclaiming

Low-state is a state stream, not a permission slip. A robot can publish state while it is not safe to move. A robot can also be in a mode that blocks high-level motion while still producing valid DDS telemetry. This distinction is central to Day 5.

Observation	Valid conclusion	Invalid conclusion
<code>rt/lowstate</code> messages arrive.	DDS observation path is working.	The robot is ready to walk.
IMU values are plausible.	Orientation telemetry is being decoded.	Balance controller is healthy for commanded motion.
Motor array exists.	Joint-state data is present in the message.	Low-level command publication is safe.
<code>mode_machine</code> has a value.	Machine state is available for interpretation.	The student understands all possible transitions.
Message rate is stable.	Streaming is continuous under current conditions.	High-level RPC clients must also work.

The instructor should deliberately present one or two ambiguous cases. For example, if lowstate streams while the robot is in damped mode, students must explain why DDS health and motion readiness diverge. Conversely, if a service call fails while lowstate works, students must recognize that topic subscription and RPC service availability are related but separate parts of the stack.

12. Lab 2 Lecture Notes: Read-Only FSM and Motion-Mode Inspection

Lab 2 introduces high-level service inspection without high-level motion. The script creates a `MotionSwitcherClient` and a `G1 LocoClient`. It uses `CheckMode()` to inspect the current motion-service ownership and uses `G1 locomotion GET` APIs to inspect finite-state-machine readiness. The course notes identify GET API IDs such as 7001 for FSM id, 7002 for FSM mode, 7003 for balance mode, 7004 for swing height, and 7005 for stand height. ¹

The key rule is that Lab 2 must remain **GET-only**. Students may inspect mode and readiness, but they should not issue standing, walking, waving, damping, or special-action commands as part of Day 5. This restraint is what prepares them to teach humanoid safety responsibly.

Read-only check	Expected teaching interpretation
<code>MotionSwitcherClient.CheckMode()</code>	Identifies which motion service or mode is active. The course readiness rule expects <code>name == 'ai'</code> for later high-level examples.
FSM id GET	Indicates the locomotion finite-state-machine state. It is one of the main readiness indicators.
FSM mode GET	Provides additional high-level state context.
Balance mode GET	Helps explain whether balance-related configuration is being reported.
Swing height GET	Demonstrates that locomotion parameters are readable before they are modified.
Stand height GET	Shows body-height configuration in a read-only manner.

Known FSM hints in the Day 5 materials include 0 for zero torque, 1 for damp, 3 for sit, 500 for start, 702 for lie-to-stand, and 706 for squat/stand transition. ¹ The instructor should present these values as course-level operational hints rather than as a complete formal FSM specification. The exact robot software version and configuration may affect behavior, so the safest teaching phrase is: **interpret FSM values using the course script, official documentation, and the observed physical state together.**

FSM hint	Course interpretation	Instructor guidance
0	Zero torque	Do not infer balance readiness. Confirm physical support and procedure.
1	Damp	Treat as not ready for high-level motion; the robot may fall if unsupported.
3	Sit	A seated posture may be safe for observation but not walking.

500	Start	Transitional or startup-related state; wait and re-check.
702	Lie-to-stand	Transition state; do not interrupt casually.
706	Squat/stand transition	Transition state; maintain distance and wait for stability.

A useful summary readiness rule for Day 5 is:

Day 5 G1 readiness rule: `rt/lowstate` must stream correctly, `CheckMode()` should report the expected high-level service ownership, and the FSM state must not indicate damp or an unsafe transitional posture before any later motion lab is considered.

13. Readiness States: FAIL, PARTIAL, and READY

The Day 5 script behavior should be taught as a diagnostic classification rather than a binary pass/fail. A system can be partially healthy: ping may work while DDS does not; DDS may work while service calls fail; service calls may work while the FSM indicates that motion is blocked. This layered interpretation is exactly what future trainers need.

Classification	Evidence pattern	Meaning	Instructor action
FAIL — local setup	SDK imports or CycloneDDS discovery fail before robot communication.	The laptop environment is not ready.	Fix installation and environment variables before touching robot state.
FAIL — network	Interface lacks <code>192.168.123.x</code> or ping fails.	The laptop cannot reach the expected robot control path.	Check cable, adapter, subnet, and robot power state.
PARTIAL — IP only	Ping works but <code>rt/lowstate</code> does not stream.	Physical/IP path exists, but DDS or topic binding is not healthy.	Inspect interface binding, DDS profile, robot mode, and IDL type.
PARTIAL — DDS only	Lowstate streams but RPC checks fail.	Topic subscription works, but service access is not proven.	Check service availability, SDK client initialization, and mode.
PARTIAL — not motion-ready	Lowstate and RPC work, but FSM	Communication is healthy, but command	Do not move; recover posture/mode using

	indicates damp or transition.	readiness is blocked.	approved procedure.
READY for Day 6 consideration	Lowstate streams, read-only RPC succeeds, expected mode is active, FSM is safe, and physical environment is clear.	The system may proceed to supervised motion labs later.	Record evidence and obtain instructor sign-off.

The word **READY** should be used carefully. In Day 5, it means “ready to proceed to the next supervised learning step,” not “ready for arbitrary student commands.” The instructor should model this language precisely.

14. G1 Versus Go2/B2: Avoiding Transfer Errors

A major Day 5 objective is to stop students from transferring quadruped assumptions into humanoid control. Go2 and B2 work from a quadruped posture and were previously used for patrol, inspection evidence, camera capture, and supervised motion. G1 introduces humanoid balance, different message namespaces, different state readiness, and stricter body-safety implications.

Topic	Go2/B2 course habit	G1 Day 5 correction
Physical posture	Stable quadruped stance or crouch can be assumed in many labs.	Humanoid balance depends on active control; damping may cause a fall.
Message namespace	Quadruped examples often use <code>unitree_go</code> .	G1 lowstate uses <code>unitree_hg</code> in the Day 5 script.
First useful task	Patrol, obstacle avoidance, or inspection evidence.	State observation and readiness proof.
Motion confidence	High-level motion can be tested after bounded checks.	No motion until mode, FSM, space, spotter, and instructor sign-off align.
Failure interpretation	A stop may result in stable quadruped behavior.	A stop-like state may produce loss of balance depending on posture and control mode.
Student excitement	Students want to see walking or gestures.	Students must learn to certify readiness before commanding walking or gestures.

The instructor can turn this comparison into a five-minute discussion. Ask students which assumption from B2 would be most dangerous on G1. Strong answers include “assuming damp is safe,” “assuming a streaming topic means motion is ready,” and “copying the wrong IDL type.”

15. Troubleshooting Guide

Troubleshooting should be taught from the bottom of the stack upward. Students often jump to code changes when the issue is an Ethernet adapter, or they reboot the robot when the issue is an unset environment variable. Day 5 is the best time to establish a calm diagnostic routine.

Symptom	Likely layer	Diagnostic question	Recommended next step
<code>ModuleNotFoundError</code> for SDK package	Python environment	Is the correct environment active?	Re-run setup, check <code>pip show</code> , and verify import path.
Could not locate <code>cyclonedds</code>	Middleware dependency	Is <code>CYCLONEDDS_HOME</code> set to the installed CycloneDDS path?	Set/export <code>CYCLONEDDS_HOME</code> or rebuild CycloneDDS as needed. ⁹
Network interface not found	OS/network	Is the adapter plugged in and named differently?	Use <code>ip addr</code> and select the interface with the robot subnet.
Ping to <code>.161</code> fails	Physical/IP	Is the laptop on <code>192.168.123.x</code> and is the robot powered?	Check cable, subnet, adapter, and robot boot state. ⁷
Ping works but lowstate times out	DDS/topic	Is DDS bound to the correct interface and using the correct G1 IDL?	Unset stale <code>CYCLONEDDS_URI</code> , verify interface, and confirm <code>unitree_hg</code> type.
Lowstate works but <code>CheckMode()</code> fails	Service/RPC	Is the high-level service reachable and initialized?	Re-run readiness script, wait after boot, inspect mode.
FSM reports damp	Robot state	Is the robot intentionally damped,	Do not command motion; use approved recovery procedure.

		unsupported, or awaiting recovery?	
Students disagree on readiness	Procedure	Which evidence is missing: lowstate, mode, FSM, physical space, or sign-off?	Return to the readiness table and classify the state.

The instructor should require students to write one sentence for every failure: **“The current evidence proves X, but does not yet prove Y.”** For example, “Ping proves IP reachability, but does not yet prove DDS discovery.” This practice prevents unsafe leaps in reasoning.

16. Instructor Demonstration Script

A disciplined instructor demonstration for Day 5 should look slow and methodical. The instructor should speak aloud at each layer and explain why the class is not moving the robot yet.

Step	Instructor action	Spoken teaching point
1	Show the physical robot, safe area, remote-control owner, and spotter.	“Before software, we establish the human safety system.”
2	Show the Ethernet connection and laptop interface list.	“The SDK needs the local interface name, not just the robot IP address.”
3	Ping the robot control address.	“Ping proves IP reachability, not DDS or motion readiness.”
4	Run the local install verification.	“This proves the laptop environment can import the SDK and find middleware.”
5	Subscribe to <code>rt/lowstate</code> .	“Now we prove robot telemetry is reaching us through DDS.”
6	Interpret <code>tick</code> , IMU, motor count, and mode fields.	“We are reading state; we are not commanding state.”

7	Run read-only <code>CheckMode()</code> and FSM GET calls.	“Now we query motion readiness without asking the robot to move.”
8	Classify the result as FAIL, PARTIAL, or READY for later supervised labs.	“Readiness is a layered decision, not a green light for arbitrary motion.”

This demonstration can be repeated by student teams. Each team should present its evidence in the same order. The uniform sequence helps future trainers reproduce Day 5 consistently across classrooms.

17. Student Lab Record Template

Students should maintain a concise but complete Day 5 lab record. The record is not just administrative; it teaches traceability. If a Day 6 motion problem occurs, the Day 5 record helps identify whether the system was genuinely ready or whether a previous assumption was never verified.

Field	Student entry
Date and team	
Robot identifier	
Laptop hostname	
Network interface name	
Laptop interface IP	
Ping to <code>192.168.123.161</code> result	
SDK import verification result	
CycloneDDS status	
<code>CYCLONEDDS_URI</code> status	
First <code>rt/lowstate</code> message time	
Observed <code>tick</code> behavior	
Observed IMU values	
Observed motor count	

CheckMode() output	
FSM id and interpretation	
Classification	FAIL / PARTIAL / READY for next supervised step
Instructor sign-off	

The instructor should remind students that “READY” requires both software evidence and physical safety evidence. A script cannot see whether a student is standing inside the fall radius.

18. Assessment Questions

The final fifteen minutes should include oral or written knowledge checks. These questions should test readiness reasoning, not memorization alone.

Question	Expected answer characteristics
Why does Day 5 avoid motion commands even though the SDK supports control?	The student should mention humanoid fall risk, active balance, debugging readiness, and the need to separate observation from command.
What is the difference between the robot IP and the SDK network-interface argument?	The robot IP is the remote address used for reachability tests; the interface argument is the local adapter name bound to the robot subnet.
What does successful <code>rt/lowstate</code> subscription prove?	It proves DDS topic observation is working with the correct topic/type; it does not prove motion readiness.
Why is using the correct IDL namespace important?	G1 uses G1/HG message types in the Day 5 lowstate lab, while quadruped examples often use Go message types. Wrong type binding can break decoding.
What does FSM = damp imply operationally?	The robot is not ready for high-level motion and may lose balance if unsupported; recovery must follow approved procedure.
If ping works but lowstate does not, what layer should be investigated next?	DDS discovery, interface binding, CycloneDDS profile, topic name, message type, and robot publishing state.

If lowstate works but service calls fail, what should students avoid concluding?	They should avoid concluding that motion is ready; service/RPC readiness is not proven.
Why is <code>Damp()</code> not a casual stop command on a standing humanoid?	Damping can remove active balance and may cause the robot to fall.

A practical assessment can be performed by asking each team to classify a provided scenario. For example: “SDK imports pass, ping passes, lowstate streams, `CheckMode()` succeeds, but FSM id is 1.” The correct classification is **PARTIAL — not motion-ready**, because damped state blocks safe high-level motion.

19. Instructor Notes for Timing and Classroom Management

Day 5 can easily run long if students experience network or installation issues. The instructor should manage the session by keeping the class aligned at shared checkpoints rather than allowing every team to debug independently for long periods. If one team fails at local imports while most teams are ready for DDS, pair that team with a teaching assistant or provide a known-good machine so the lecture objective is not lost.

Risk to timing	Mitigation
Students spend too long installing dependencies.	Use preflight setup before class and keep a known-good laptop available.
Interface confusion consumes lab time.	Demonstrate <code>ip addr</code> slowly and require everyone to identify the adapter before running scripts.
Robot not fully booted before tests.	Build a waiting period into the safety checklist and do not interpret early failures too quickly.
Students want to send a motion command.	Repeat the Day 5 contract: observation and readiness only; motion belongs to Day 6.
One group changes DDS settings and affects later tests.	Standardize environment variables at the start and record whether <code>CYCLONEDDS_URI</code> is set.
Multiple students approach the robot during troubleshooting.	Keep one operator, one spotter, and a clear perimeter.

The instructor's tone matters. Day 5 should not feel like a restriction; it should feel like professional robotics practice. The best trainers make readiness checks feel as important as successful motion.

20. Day 6 Preview

Day 6 will build on Day 5 by introducing controlled G1 high-level actions and, depending on the course branch, arm or gesture-related examples. The Day 5 deliverable is therefore a readiness certificate. If students cannot explain their Day 5 evidence, they are not prepared to responsibly teach Day 6.

The transition statement should be:

“Tomorrow we will only command what today we learned to observe, classify, and verify.”

This phrasing reinforces the course's progression from state awareness to command authority. It also helps trainers communicate safety expectations to future student cohorts.

21. Key Takeaways

Day 5 is the foundation of responsible G1 teaching. Students learn that a humanoid platform requires a stricter readiness mindset than a quadruped inspection platform. They learn that network reachability, DDS streaming, and motion-service readiness are separate layers. They learn that `rt/lowstate` proves observation, not motion safety. They learn that read-only FSM queries are a professional step before control. Most importantly, they learn that a powerful humanoid robot must be approached through evidence, procedure, and restraint.

Takeaway	Trainer wording
Observation precedes command.	“If we cannot read the robot safely, we do not command it.”
Ping is not DDS.	“IP reachability is one layer, not the whole stack.”
DDS is not motion readiness.	“Streaming state is evidence, not permission.”
Mode matters.	“A service mode or FSM state can block safe motion even when communication works.”

Humanoid damping is serious.	“Damp can mean fall risk, not merely pause.”
G1 is not Go2/B2.	“Use the right IDL, the right posture assumptions, and the right safety procedure.”

References

- [1] Vinci Unitree Day 5 README
- [2] Vinci Unitree Train-the-Trainer Schedule
- [3] Vinci Unitree G1 Field Guide
- [4] Vinci Unitree G1 vs Go2 Notes
- [5] Unitree G1 SDK Development Guide — About G1
- [6] Unitree G1 SDK Development Guide — Quick Start
- [7] Unitree G1 SDK Development Guide — Quick Development
- [8] Unitree G1 Product Page
- [9] Unitree SDK2 Python Repository
- [10] Unitree SDK Development Guide — Quick Start
- [11] Unitree Sports Services Interface

Prepared for Vinci AI Unitree Robotics Training

Author: Manus AI

Document type: Comprehensive 3-hour lecture note